# Parallel & Distributed Computing

## UCS645

## Assignment-2

Name: Kartik Juneja

Roll No: 102303274

**System Environment:** Linux(Fedora), G++ compiler

**Aim:**

- Implement basic OpenMP parallel programs.
- Measure execution time using omp_get_wtime().
- Compute speedup, efficiency, and cost metrics.
- Understand strong vs weak scaling using Amdahl's and Gustafson's
- laws.
- Identify performance bottlenecks such as load imbalance,
- synchronization overhead, false sharing, and memory bandwidth
- saturation.
- Gain introductory exposure to performance profiling tools.

**Q1.** Molecular Dynamics

**CODE:**

```
#include <chrono>
#include <iostream>
#include <ostream>
#include <random>
#include <vector>
#include <format>
#include <omp.h>
```

```
#define SQUARE(a) (a*a)
#define TOTAL_PARTICLES 1e4

constexpr double EPSILON{1.0};
constexpr double SIGMA{1.0};
constexpr double SIGMASQ{SQUARE(SIGMA)};
struct vec3_t {
	double x{},y{},z{};

	vec3_t() = default;
	vec3_t(double x, double y, double z): x{x}, y{y}, z{z} {}

	vec3_t operator+(const vec3_t& other) const { return {x + other.x, y + other.y, z + other.z}; }
	vec3_t operator-(const vec3_t& other) const { return {x - other.x, y - other.y, z - other.z}; }
	vec3_t& operator+=(const vec3_t& other) {
		x += other.x;
		y += other.y;
		z += other.z;
		return *this;
	}
	vec3_t& operator-=(const vec3_t& other) {
		x -= other.x;
		y -= other.y;
		z -= other.z;
		return *this;
	}
	vec3_t operator*(double scalar) const { return {x * scalar, y * scalar, z * scalar}; }
	friend auto operator<<(std::ostream& os, const vec3_t& obj) -> std::ostream&;
};
```

```
auto operator<<(std::ostream& os, const vec3_t& obj) -> std::ostream& {
	os << std::format("({:.3f})i + ({:.3f})j + ({:.3f})k\n", obj.x, obj.y, obj.z);
	return os;
}

struct Particles {
	std::vector<vec3_t> position{};
	std::vector<vec3_t> force{};
};

auto get_random_number() -> double {
	static thread_local std::mt19937 mt{std::random_device{}()};
	static thread_local std::uniform_real_distribution<double> range{-5.0, 5.0};
	return range(mt);
}

auto generate_random_vector() -> vec3_t {
	vec3_t random_vector{get_random_number(), get_random_number(), get_random_number()};
	return random_vector;
}

auto init_particles(Particles& particles) -> void {
	int n{static_cast<int>(particles.position.size())};
	#pragma omp parallel for
	for(int i = 0; i<n; ++i){
		particles.position[i] = generate_random_vector();
		particles.force[i] = {0.0, 0.0, 0.0};
	}
};
```

```
auto compute_potential_and_force_sequential(Particles& particles, double& total_energy) ->
std::chrono::duration<double, std::milli> {
    total_energy = 0.0;
    int n{static_cast<int>(particles.position.size())};

    std::cout << "Using No Threading\n";
    auto no_start{std::chrono::steady_clock::now()};
    for(int i{0}; i<n;++i) {
        vec3_t current_force{0.0, 0.0, 0.0};
        for(int j{0}; j<n; ++j) {
            if(i == j) continue;
            vec3_t delta{particles.position[i] - particles.position[j]};

            double r2{SQUARE(delta.x) + SQUARE(delta.y) + SQUARE(delta.z)};

            if (r2 < 1e-10) continue;
            double r2_inv{1.0/r2};
            double s2_inv{SIGMASQ*r2_inv};
            double s6_inv{SQUARE(s2_inv) * s2_inv};
            double s12_inv{SQUARE(s6_inv)};

            double pair_energy{4 * EPSILON * (s12_inv - s6_inv)};
            total_energy += pair_energy;

            double force_scalar{(24.0 * EPSILON * r2_inv) * (2.0 * s12_inv - s6_inv)};
            vec3_t force_vec{delta * force_scalar};

            current_force += force_vec;
        }
        particles.force[i] = current_force;
```

```
    }
    auto no_end{std::chrono::steady_clock::now()};
    std::chrono::duration<double, std::milli> ms{no_end - no_start};
    std::cout << std::format("Execution time: {:.2f}ms\n", ms.count());
    std::cout << std::format("Total Energy: {}\n", total_energy * 0.5);
    return ms;
}

auto compute_potential_and_force_parallel(Particles& particles, double& total_energy, int
num_threads) -> std::chrono::duration<double, std::milli> {
    total_energy = 0.0;
    int n{static_cast<int>(particles.position.size())};

    #pragma omp parallel for
    for(int i = 0; i<n; ++i){
        particles.force[i] = {0.0, 0.0, 0.0};
    }

    total_energy = 0.0;
    std::cout << std::format("With {} threads\n", num_threads);
    auto th_start{std::chrono::steady_clock::now()};
    #pragma omp parallel for reduction(+:total_energy) schedule(dynamic)
num_threads(num_threads)
    for(int i = 0; i<n;++i) {
        vec3_t current_force{0.0, 0.0, 0.0};
        for(int j = 0; j<n; ++j) {
            if(i == j) continue;
            vec3_t delta{particles.position[i] - particles.position[j]};

            double r2{SQUARE(delta.x) + SQUARE(delta.y) + SQUARE(delta.z)};
```

```
                if (r2 < 1e-10) continue;
                double r2_inv{1.0/r2};
                double s2_inv{SIGMASQ*r2_inv};
                double s6_inv{SQUARE(s2_inv) * s2_inv};
                double s12_inv{SQUARE(s6_inv)};

                double pair_energy{4 * EPSILON * (s12_inv - s6_inv)};
                total_energy += pair_energy;

                double force_scalar{(24.0 * EPSILON * r2_inv) * (2.0 * s12_inv - s6_inv)};
                vec3_t force_vec{delta * force_scalar};

                current_force += force_vec;
            }
            particles.force[i] = current_force;
        }
        auto th_end{std::chrono::steady_clock::now()};
        std::chrono::duration<double, std::milli> ms{th_end - th_start};
        std::cout << std::format("Execution time: {:.2f}ms\n", ms.count());
        std::cout << std::format("Total Energy: {}\n", total_energy * 0.5);
        return ms;
}

auto main(void) -> int {
        Particles particles{};
        particles.position.resize(TOTAL_PARTICLES);
        particles.force.resize(TOTAL_PARTICLES);
        init_particles(particles);
        int max_threads{omp_get_max_threads()};
```

```
    double total_energy{0.0};
    auto seq_ms{compute_potential_and_force_sequential(particles, total_energy)};
    std::cout << '\n';
    for(int num_threads{2}; num_threads <= max_threads; num_threads += 2) {
        auto par_ms{compute_potential_and_force_parallel(particles, total_energy, num_threads)};
        double speed_up{seq_ms.count()/par_ms.count()};
        std::cout << std::format("Speed Up: {:.2f}x\n", speed_up);
        std::cout << std::format("Throughput: {:.2f}\n", TOTAL_PARTICLES/par_ms.count());
        std::cout << std::format("Efficiency: {:.2f}\n\n", speed_up/num_threads);
    }
    return 0;
}
```

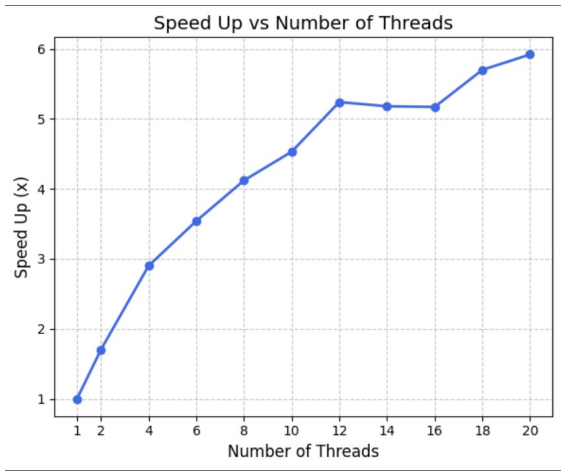
**OUTPUT:**

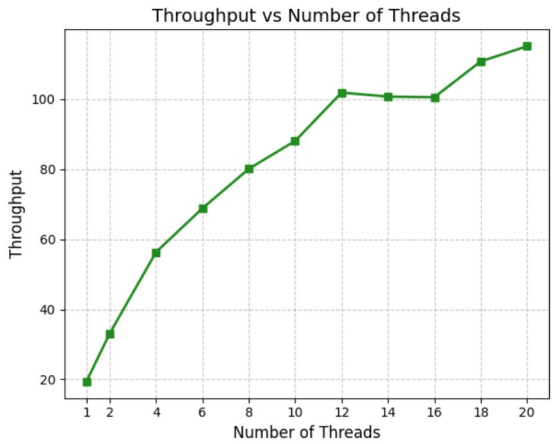
| Threads | Execution Time (ms) | Speed Up | Throughput | Efficiency |
|---|---|---|---|---|
| No Threading | 514.52 | - | - | - |
| 2 | 301.90 | 1.70x | 33.12 | 0.85 |
| 4 | 177.49 | 2.90x | 56.34 | 0.72 |
| 6 | 145.26 | 3.54x | 68.84 | 0.59 |
| 8 | 124.84 | 4.12x | 80.10 | 0.52 |
| 10 | 113.53 | 4.53x | 88.08 | 0.45 |
| 12 | 98.17 | 5.24x | 101.86 | 0.44 |
| 14 | 99.29 | 5.18x | 100.72 | 0.37 |
| 16 | 99.47 | 5.17x | 100.53 | 0.32 |
| 18 | 90.28 | 5.70x | 110.76 | 0.32 |
| 20 | 86.88 | 5.92x | 115.10 | 0.30 |

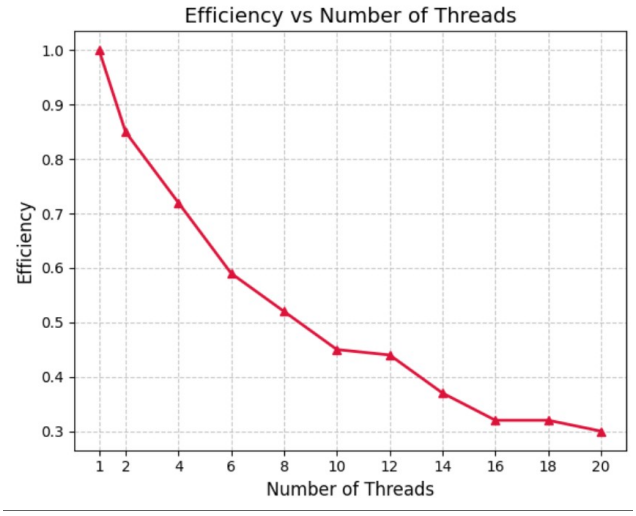
**OBSERVATION:**

A molecular dynamics simulation was parallelized using OpenMP across 10,000 particles, computing pairwise forces and potential energies for every particle interaction — an $O(N^2)$ workload. The reduction(+:total_energy) clause safely accumulates energy across threads, and schedule(dynamic) is used to distribute iterations at runtime, which helps balance load since particle pairs at different distances involve varying computation. The sequential baseline ran at 514.52ms. Speedup improved steadily with thread count, reaching a peak of 5.92× at 20 threads, though efficiency dropped consistently from 0.85 at 2 threads down to 0.30 at 20 threads. A notable plateau is observed between

12 and 16 threads (~5.17–5.24×), suggesting the workload saturates available memory bandwidth or hits synchronization overhead before recovering slightly at 18 and 20 threads due to dynamic scheduling filling idle slots more effectively.



**OBSERVATION FROM PERF:**

```
Performance counter stats for './outputs/molecular':

            1,015      context-switches                 #     84.5 cs/sec  cs_per_second
              273      cpu-migrations                   #     22.7 migrations/sec  migrations_per_second
              523      page-faults                      #     43.5 faults/sec  page_faults_per_second
       12,012.94 msec task-clock                       #      6.4 CPUs  CPUs_utilized
          520,745      cpu_core/branch-misses/          #      0.0 %  branch_miss_rate         (75.80%)
    3,302,257,377      cpu_core/branches/               #    274.9 M/sec  branch_frequency     (75.80%)
   13,818,541,079      cpu_core/cpu-cycles/             #      1.2 GHz  cycles_frequency       (75.80%)
   35,376,687,078      cpu_core/instructions/           #      2.6 instructions  insn_per_cycle  (75.80%)
          376,510      cpu_atom/branch-misses/          #      0.0 %  branch_miss_rate         (27.96%)
    1,801,366,423      cpu_atom/branches/               #    150.0 M/sec  branch_frequency     (27.22%)
   10,975,658,458      cpu_atom/cpu-cycles/             #      0.9 GHz  cycles_frequency       (27.15%)
   19,050,273,691      cpu_atom/instructions/           #      1.7 instructions  insn_per_cycle  (27.21%)
            TopdownL1 (cpu_core)                       #      0.9 %  tma_bad_speculation
                                                       #      6.0 %  tma_frontend_bound       (75.80%)
                                                       #     44.1 %  tma_backend_bound
                                                       #     49.0 %  tma_retiring             (75.80%)
            TopdownL1 (cpu_atom)                       #     64.5 %  tma_backend_bound        (31.35%)
                                                       #      0.4 %  tma_frontend_bound       (31.42%)
                                                       #      1.3 %  tma_bad_speculation
                                                       #     33.7 %  tma_retiring             (32.50%)

      1.871037079 seconds time elapsed

     11.881381000 seconds user
      0.044567000 seconds sys
```

The program utilized 6.4 CPUs on average across the full run, reflecting the blend of sequential and parallel phases measured together. The IPC of 2.6 on cpu_core is strong, indicating the Lennard-Jones force computation keeps the pipeline well utilized with its repeated arithmetic operations. The branch miss rate is effectively 0%, which is expected since the inner loop has minimal conditional branching. The TopdownL1 analysis shows 44.1% backend-bound on cpu_core and 64.5% on cpu_atom, pointing to memory latency as the primary bottleneck — consistent with the $O(N^2)$ access pattern where each particle must read the positions of all others, stressing the cache heavily and explaining the efficiency drop at higher thread counts.

**Q2.** DNA Sequence Alignment

**CODE:**

```
#include <chrono>
#include <iostream>
#include <random>
#include <vector>
#include <algorithm>
#include <string>
#include <omp.h>
constexpr int SEQUQNCE_LENGTH{static_cast<int>(1e4)};
constexpr int MATCH{3};
constexpr int MISMATCH{-3};
constexpr int GAP{-2};
constexpr char POSSIBLE[4] {'A', 'T', 'G', 'C'};
constexpr int tile_sizes[7] {32, 64, 96, 128, 192, 256, 512};
auto get_random_number() -> int {
	static thread_local std::mt19937 mt{std::random_device{}()};
	static thread_local std::uniform_int_distribution<int> range{0, 3};
	return range(mt);
}
auto generate_random_sequence() -> std::string {
	std::string str(SEQUQNCE_LENGTH, 'A');
	#pragma omp parallel for
	for(int i = 0; i<SEQUQNCE_LENGTH; ++i) {
		str[i] = POSSIBLE[get_random_number()];
	}
	return str;
}
auto process_tile(int block_row, int block_col, const std::string& seqA, const std::string& seqB, int
tile_size,
```

```
                std::vector<std::vector<int>>& score_matrix, int& global_max_score) -> void {
    int start_i{block_row * tile_size + 1};
    int start_j{block_col * tile_size + 1};
    int end_i{std::min((int)seqA.length() + 1, start_i + tile_size)};
    int end_j{std::min((int)seqB.length() + 1, start_j + tile_size)};
    int local_max{0};
    for (int i = start_i; i < end_i; ++i) {
        for (int j = start_j; j < end_j; ++j) {

            int score_diag = score_matrix[i-1][j-1] + ((seqA[i-1] == seqB[j-1]) ? MATCH : MISMATCH);
            int score_up   = score_matrix[i-1][j] + GAP;
            int score_left = score_matrix[i][j-1] + GAP;
            int current_score = std::max({0, score_diag, score_up, score_left});
            score_matrix[i][j] = current_score;
            local_max = std::max(local_max, current_score);
        }
    }
    if (local_max > global_max_score) {
        #pragma omp critical
        {
            if (local_max > global_max_score) {
                global_max_score = local_max;
            }
        }
    }
}
auto dna_sequqnce_allignment_sequential(const std::string& seqA, const std::string& seqB) ->
std::chrono::duration<double, std::milli> {
        int rows{static_cast<int>(seqA.length() + 1)};
        int cols{static_cast<int>(seqB.length() + 1)};
```

```
    std::vector<int> prev_row(cols, 0);
    std::vector<int> curr_row(cols, 0);
    int max_score{0};
    auto start{std::chrono::steady_clock::now()};
    for (int i = 1; i < rows; ++i) {
        for (int j = 1; j < cols; ++j) {
            int score_diag = prev_row[j-1] +
                ((seqA[i-1] == seqB[j-1]) ? MATCH : MISMATCH);
            int score_up = prev_row[j] + GAP;
            int score_left = curr_row[j-1] + GAP;
            int current_val = std::max({0, score_diag, score_up, score_left});
            curr_row[j] = current_val;
            max_score = std::max(max_score, current_val);
        }
        prev_row = curr_row;
        std::fill(curr_row.begin(), curr_row.end(), 0);
    }
    auto end{std::chrono::steady_clock::now()};
    std::chrono::duration<double, std::milli> ms{end-start};
    std::cout << std::format("Score: {}\n", max_score);
    std::cout << std::format("Execution time: {:.2f}ms\n", ms.count());
    return ms;
}
auto dna_sequence_allignment_parallel(const std::string& seqA, const std::string& seqB, int tile_size)
-> std::chrono::duration<double, std::milli> {
    int rows{static_cast<int>(seqA.length() + 1)};
    int cols{static_cast<int>(seqB.length() + 1)};
    std::vector<std::vector<int>> matrix(rows, std::vector<int>(cols, 0));
    int n_block_rows{(rows - 1 + tile_size - 1) / tile_size};
    int n_block_cols{(cols - 1 + tile_size - 1) / tile_size};
```

```
std::vector<int> deps(n_block_rows * n_block_cols, 0);
int dummy_root = 0;
int global_max_score{0};
auto start{std::chrono::steady_clock::now()};
#pragma omp parallel
{
    #pragma omp single
    {
        for (int i = 0; i < n_block_rows; ++i) {
            for (int j = 0; j < n_block_cols; ++j) {
                int self_idx = i * n_block_cols + j;
                int* self = &deps[self_idx];
                int* top = (i == 0) ? &dummy_root : &deps[(i-1) * n_block_cols + j];
                int* left = (j == 0) ? &dummy_root : &deps[i * n_block_cols + (j-1)];
                #pragma omp task \
                depend(in: *top) \
                depend(in: *left) \
                depend(out: *self)
                {
                    process_tile(i, j, seqA, seqB, tile_size, matrix, global_max_score);
                }
            }
        }
    }
}
auto end{std::chrono::steady_clock::now()};
std::chrono::duration<double, std::milli> ms{end-start};
std::cout << std::format("Score: {}\n", global_max_score);
std::cout << std::format("Execution time: {:.2f}ms\n", ms.count());
return ms;
```

```
}
auto main() -> int {
	std::string seqA{generate_random_sequence()};
	std::string seqB{generate_random_sequence()};
	std::cout << "With No thearding\n";
	auto seq_time{dna_sequqnce_allignment_sequential(seqA, seqB)};
	int max_threads{omp_get_max_threads()};
	std::cout << "\nWavefront optimized\n";
	for(int num_threads{2}; num_threads<max_threads; num_threads += 2) {
		omp_set_num_threads(num_threads);
		std::cout << std::format("With {} threads\n", num_threads);
		for(int  i{0}; i<7;++i) {
			std::cout << std::format("{} x {} tile size\n", tile_sizes[i], tile_sizes[i]);
			auto par_time{dna_sequence_allignment_parallel(seqA, seqB, tile_sizes[i])};
			auto speed_up{seq_time.count()/par_time.count()};
			std::cout << std::format("Speed Up: {:.2f}x\n", speed_up);
			std::cout << std::format("Throughput: {:.2f}\n",
(SEQUQNCE_LENGTH*SEQUQNCE_LENGTH)/par_time.count());
			std::cout << std::format("Efficiency: {:.2f}\n\n", speed_up/num_threads);
		}
	}
	return 0;
}
```

**OUTPUT:**

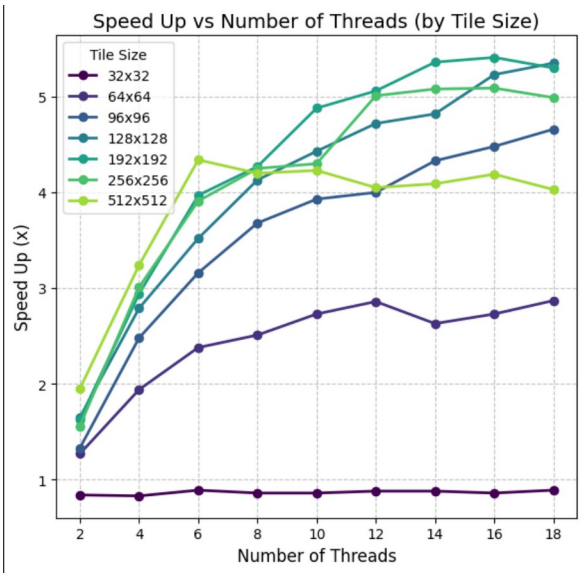
| Threads | Tile Size | Execution Time (ms) | Speed Up | Throughput | Efficiency |
|---|---|---|---|---|---|
| No Threading | - | 229.21 | - | - | - |
| 2 | 32 x 32 | 271.93 | 0.84x | 367,735.11 | 0.42 |
| | 64 x 64 | 180.94 | 1.27x | 552,664.31 | 0.63 |
| | 96 x 96 | 172.06 | 1.33x | 581,193.00 | 0.67 |
| | 128 x 128 | 138.57 | 1.65x | 721,648.59 | 0.83 |
| | 192 x 192 | 140.56 | 1.63x | 711,432.86 | 0.82 |
| | 256 x 256 | 146.90 | 1.56x | 680,737.85 | 0.78 |
| | 512 x 512 | 117.77 | 1.95x | 849,085.00 | 0.97 |
| 4 | 32 x 32 | 275.63 | 0.83x | 362,805.56 | 0.21 |
| | 64 x 64 | 117.87 | 1.94x | 848,425.12 | 0.49 |
| | 96 x 96 | 92.34 | 2.48x | 1,083,001.26 | 0.62 |
| | 128 x 128 | 82.02 | 2.79x | 1,219,150.03 | 0.70 |
| | 192 x 192 | 77.91 | 2.94x | 1,283,594.92 | 0.74 |
| | 256 x 256 | 76.04 | 3.01x | 1,315,058.11 | 0.75 |
| | 512 x 512 | 70.67 | 3.24x | 1,415,041.23 | 0.81 |
| 6 | 32 x 32 | 257.55 | 0.89x | 388,277.84 | 0.15 |
| | 64 x 64 | 96.32 | 2.38x | 1,038,190.47 | 0.40 |
| | 96 x 96 | 72.59 | 3.16x | 1,377,564.33 | 0.53 |
| | 128 x 128 | 65.16 | 3.52x | 1,534,574.06 | 0.59 |
| | 192 x 192 | 57.79 | 3.97x | 1,730,280.34 | 0.66 |
| | 256 x 256 | 58.63 | 3.91x | 1,705,706.59 | 0.65 |
| | 512 x 512 | 52.78 | 4.34x | 1,894,762.29 | 0.72 |
| 8 | 32 x 32 | 267.78 | 0.86x | 373,445.83 | 0.11 |
| | 64 x 64 | 91.48 | 2.51x | 1,093,159.55 | 0.31 |
| | 96 x 96 | 62.21 | 3.68x | 1,607,336.22 | 0.46 |
| | 128 x 128 | 55.51 | 4.13x | 1,801,504.80 | 0.52 |
| | 192 x 192 | 53.71 | 4.27x | 1,861,765.17 | 0.53 |
| | 256 x 256 | 53.90 | 4.25x | 1,855,158.67 | 0.53 |

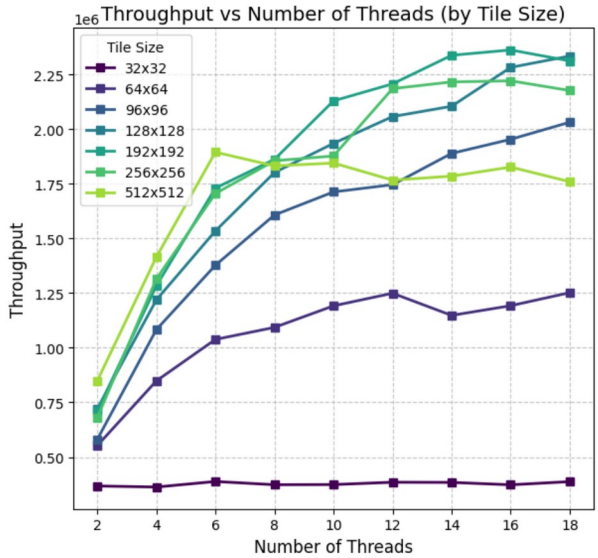
| Threads | Tile Size | Execution Time (ms) | Speed Up | Throughput | Efficiency |
|---|---|---|---|---|---|
| | 512 x 512 | 54.59 | 4.20x | 1,831,871.39 | 0.52 |
| 10 | 32 x 32 | 267.29 | 0.86x | 374,131.64 | 0.09 |
| | 64 x 64 | 83.91 | 2.73x | 1,191,754.57 | 0.27 |
| | 96 x 96 | 58.38 | 3.93x | 1,712,832.41 | 0.39 |
| | 128 x 128 | 51.68 | 4.43x | 1,934,811.29 | 0.44 |
| | 192 x 192 | 46.96 | 4.88x | 2,129,310.61 | 0.49 |
| | 256 x 256 | 53.27 | 4.30x | 1,877,357.88 | 0.43 |
| | 512 x 512 | 54.20 | 4.23x | 1,844,924.47 | 0.42 |
| 12 | 32 x 32 | 259.88 | 0.88x | 384,794.31 | 0.07 |
| | 64 x 64 | 80.04 | 2.86x | 1,249,358.95 | 0.24 |
| | 96 x 96 | 57.28 | 4.00x | 1,745,906.71 | 0.33 |
| | 128 x 128 | 48.60 | 4.72x | 2,057,560.59 | 0.39 |
| | 192 x 192 | 45.31 | 5.06x | 2,207,178.83 | 0.42 |
| | 256 x 256 | 45.76 | 5.01x | 2,185,491.68 | 0.42 |
| | 512 x 512 | 56.60 | 4.05x | 1,766,804.37 | 0.34 |
| 14 | 32 x 32 | 260.38 | 0.88x | 384,057.35 | 0.06 |
| | 64 x 64 | 87.10 | 2.63x | 1,148,040.70 | 0.19 |
| | 96 x 96 | 52.94 | 4.33x | 1,889,021.75 | 0.31 |
| | 128 x 128 | 47.51 | 4.82x | 2,104,688.47 | 0.34 |
| | 192 x 192 | 42.77 | 5.36x | 2,337,823.49 | 0.38 |
| | 256 x 256 | 45.13 | 5.08x | 2,216,010.74 | 0.36 |
| | 512 x 512 | 56.02 | 4.09x | 1,785,060.41 | 0.29 |
| 16 | 32 x 32 | 268.03 | 0.86x | 373,093.07 | 0.05 |
| | 64 x 64 | 83.87 | 2.73x | 1,192,357.55 | 0.17 |
| | 96 x 96 | 51.19 | 4.48x | 1,953,459.30 | 0.28 |
| | 128 x 128 | 43.81 | 5.23x | 2,282,453.38 | 0.33 |
| | 192 x 192 | 42.33 | 5.41x | 2,362,272.43 | 0.34 |
| | 256 x 256 | 45.02 | 5.09x | 2,221,148.27 | 0.32 |
| | 512 x 512 | 54.75 | 4.19x | 1,826,564.39 | 0.26 |

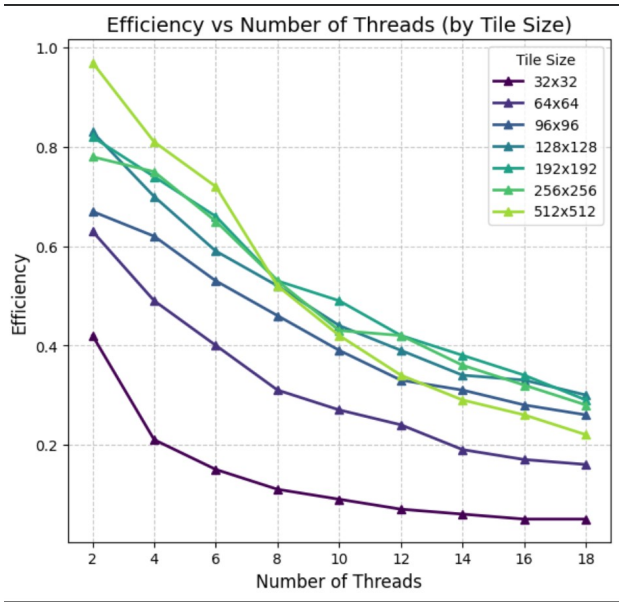
| Threads | Tile Size | Execution Time (ms) | Speed Up | Throughput | Efficiency |
|---|---|---|---|---|---|
| 18 | 32 x 32 | 257.97 | 0.89x | 387,637.99 | 0.05 |
| | 64 x 64 | 79.87 | 2.87x | 1,252,011.31 | 0.16 |
| | 96 x 96 | 49.22 | 4.66x | 2,031,593.93 | 0.26 |
| | 128 x 128 | 42.84 | 5.35x | 2,334,428.61 | 0.30 |
| | 192 x 192 | 43.25 | 5.30x | 2,312,190.64 | 0.29 |
| | 256 x 256 | 45.94 | 4.99x | 2,176,723.71 | 0.28 |
| | 512 x 512 | 56.83 | 4.03x | 1,759,709.86 | 0.22 |

**OBSERVATION:**

A wavefront-based DNA sequence alignment was parallelized using OpenMP, experimenting across thread counts (2–18) and tile sizes (32×32 to 512×512). The score remains consistent at 7175 across all configurations, confirming correctness. Small tile sizes like 32×32 consistently performed worse than sequential (speedup ~0.86×) across all thread counts, as the synchronization overhead between wavefront diagonals outweighs the benefit of parallelism for such fine-grained tiles. Performance generally improved with larger tile sizes, with the sweet spot shifting as threads increased — for example, 2 threads peaked at 512×512 (1.95×), while higher thread counts favored mid-range tiles like 192×192 or 256×256. The best overall speedup of 5.41× was achieved with 16 threads at 192×192, beyond which adding more threads yielded diminishing returns due to increasing synchronization overhead and reduced efficiency, which dropped as low as 0.05 at 32×32 with 16+ threads.






## OBSERVATION FROM PERF:

```
Performance counter stats for './outputs/dna':

          280,130      context-switches                  #   6932.5 cs/sec  cs_per_second
            1,661      cpu-migrations                    #     41.1 migrations/sec  migrations_per_second
          119,347      page-faults                       #   2953.5 faults/sec  page_faults_per_second
       40,408.11 msec task-clock                        #      4.0 CPUs  CPUs_utilized
       31,786,797      cpu_core/branch-misses/          #      0.4 %  branch_miss_rate         (72.78%)
    8,472,638,588      cpu_core/branches/               #    209.7 M/sec  branch_frequency     (72.78%)
   54,076,563,654      cpu_core/cpu-cycles/             #      1.3 GHz  cycles_frequency       (72.78%)
  144,755,358,767      cpu_core/instructions/           #      2.7 instructions  insn_per_cycle  (72.78%)
       37,407,016      cpu_atom/branch-misses/          #      0.6 %  branch_miss_rate         (17.22%)
    6,102,460,938      cpu_atom/branches/               #    151.0 M/sec  branch_frequency     (17.17%)
   47,351,353,829      cpu_atom/cpu-cycles/             #      1.2 GHz  cycles_frequency       (17.33%)
  106,070,588,819      cpu_atom/instructions/           #      2.2 instructions  insn_per_cycle  (17.42%)
           TopdownL1 (cpu_core)                         #      3.0 %  tma_bad_speculation
                                                        #      9.3 %  tma_frontend_bound       (72.78%)
                                                        #     41.5 %  tma_backend_bound
                                                        #     46.2 %  tma_retiring             (72.78%)
           TopdownL1 (cpu_atom)                         #     47.6 %  tma_backend_bound        (20.18%)
                                                        #      4.1 %  tma_frontend_bound       (20.09%)
                                                        #      2.4 %  tma_bad_speculation
                                                        #     45.9 %  tma_retiring             (20.14%)

     10.100058053 seconds time elapsed

     34.344178000 seconds user
      5.526166000 seconds sys
```

The program utilized around 4.0 CPUs on average across the full run, reflecting the mixed performance of different thread and tile configurations tested together. The IPC of 2.7 on cpu_core is strong, indicating good pipeline utilization for the dynamic programming computation involved in alignment. The branch miss rate is low at 0.4%, showing the alignment logic follows a predictable control flow. The TopdownL1 analysis shows 41.5% backend-bound on cpu_core and 47.6% on cpu_atom, pointing to memory access latency as the dominant bottleneck — which is expected given the irregular memory access patterns inherent to wavefront traversal across tile boundaries.

## **Q3.** Heat Diffusion

**CODE:**

```
#include <iostream>

#include <vector>

#include <cmath>

#include <omp.h>

#include <chrono>

#include <iomanip>

#include <string>

#include <format>

constexpr int N{2000};

constexpr int STEPS{500};

constexpr double ALPHA{0.01};

constexpr double DX{1.0};

constexpr double DT{0.1};

constexpr int CHUNK_SIZE{64};

struct SimResult {

	double duration_ms;

	double total_energy;

};

struct ScheduleConfig {

	omp_sched_t type;

	int chunk_size;
```

```
    std::string name;
};
auto run_simulation(int num_threads, omp_sched_t sched_type, int chunk_size) -> SimResult {
    std::vector<double> T(N * N, 0.0);
    std::vector<double> T_next(N * N, 0.0);
    int center{N / 2};
    int radius{N / 10};
    #pragma omp parallel for schedule(static) num_threads(num_threads)
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            if ((i - center)*(i - center) + (j - center)*(j - center) < radius*radius) {
                T[i * N + j] = 100.0;
            } else {
                T[i * N + j] = 0.0;
            }
        }
    }
    double cx {ALPHA * DT / (DX * DX)};
    double cy {ALPHA * DT / (DX * DX)};
    omp_set_num_threads(num_threads);
    omp_set_schedule(sched_type, chunk_size);
    auto start_time{std::chrono::high_resolution_clock::now()};
    for (int step{0}; step < STEPS; ++step) {
    #pragma omp parallel for schedule(runtime)
        for (int i = 1; i < N - 1; ++i) {
            for (int j = 1; j < N - 1; ++j) {
                int idx = i * N + j;
                int up = (i - 1) * N + j;
                int down = (i + 1) * N + j;
                int left = i * N + (j - 1);
```

```
                int right = i * N + (j + 1);
                T_next[idx] = T[idx] +
                    cx * (T[up] - 2 * T[idx] + T[down]) +
                    cy * (T[left] - 2 * T[idx] + T[right]);
            }
        }
        std::swap(T, T_next);
    }
    auto end_time{std::chrono::high_resolution_clock::now()};
    std::chrono::duration<double, std::milli> duration{end_time - start_time};
    double total_energy{0.0};
    #pragma omp parallel for reduction(+:total_energy) num_threads(num_threads)
    for (int i = 0; i < N * N; ++i) {
        total_energy += T[i];
    }
    return { duration.count(), total_energy };
}
auto main() -> int {
    std::cout << "With No Threading\n";
    SimResult seq_res = run_simulation(1, omp_sched_static, 0);
    std::cout << std::format("Execution time: {:.2f}ms\n", seq_res.duration_ms);
    std::cout << std::format("Total Energy: {:.4e}\n", seq_res.total_energy);
    std::vector<ScheduleConfig> strategies {
        {omp_sched_static, 0, "STATIC"},
        {omp_sched_dynamic, CHUNK_SIZE, std::format("DYNAMIC (Chunk {})",
CHUNK_SIZE)},
        {omp_sched_guided, CHUNK_SIZE, std::format("GUIDED (Chunk {})", CHUNK_SIZE)}
    };
    int max_threads{omp_get_max_threads()};
    if (max_threads < 2) max_threads = 12;
```

```
    for (int t{2}; t <= max_threads; t += 2) {
        std::cout << std::format("\nWith {} threads\n", t);
        for (const auto& strat : strategies) {
            SimResult par_res{run_simulation(t, strat.type, strat.chunk_size)};
            double speed_up{seq_res.duration_ms / par_res.duration_ms};
            double ops = (double)N * N * STEPS;
            double throughput = ops / (par_res.duration_ms / 1000.0) / 1e6;
            double efficiency{speed_up / t};
            std::cout << std::format("[ {:<20} ]\n", strat.name);
            std::cout << std::format("Execution time: {:.2f}ms\n", par_res.duration_ms);
            std::cout << std::format("Speed Up:{:.2f}x\n", speed_up);
            std::cout << std::format("Throughput: {:.2f}\n", throughput);
            std::cout << std::format("Efficiency: {:.2f}\n", efficiency);
        }
    }
    return 0;
}
```

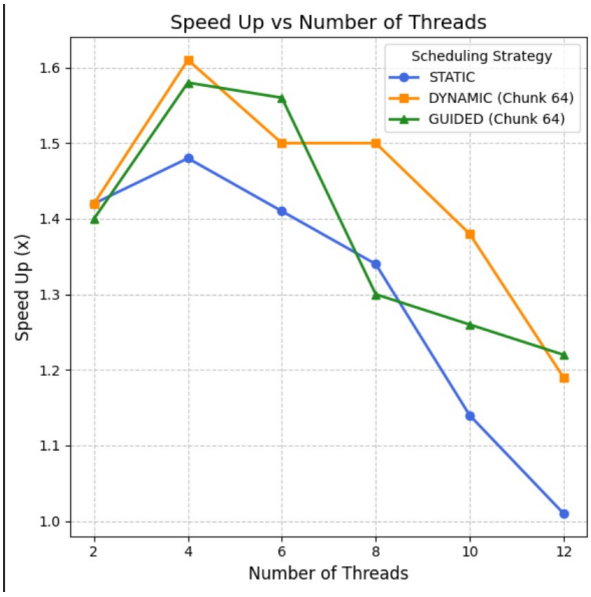
**OUTPUT:**

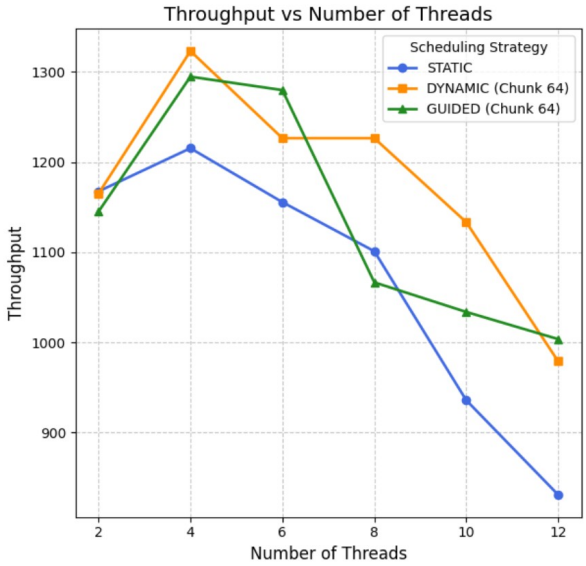
| Threads | Schedule Type | Execution Time (ms) | Speed Up | Throughput | Efficiency |
|---|---|---|---|---|---|
| **1 (No Threading)** | - | 2438.33 | - | - | - |
| **2** | STATIC | 1713.12 | 1.42x | 1167.46 | 0.71 |
| | DYNAMIC (64) | 1716.76 | 1.42x | 1164.99 | 0.71 |
| | GUIDED (64) | 1746.76 | 1.40x | 1144.98 | 0.70 |
| **4** | STATIC | 1645.62 | 1.48x | 1215.35 | 0.37 |
| | DYNAMIC (64) | 1511.61 | 1.61x | 1323.10 | 0.40 |
| | GUIDED (64) | 1544.80 | 1.58x | 1294.67 | 0.39 |
| **6** | STATIC | 1730.76 | 1.41x | 1155.56 | 0.23 |
| | DYNAMIC (64) | 1630.66 | 1.50x | 1226.50 | 0.25 |

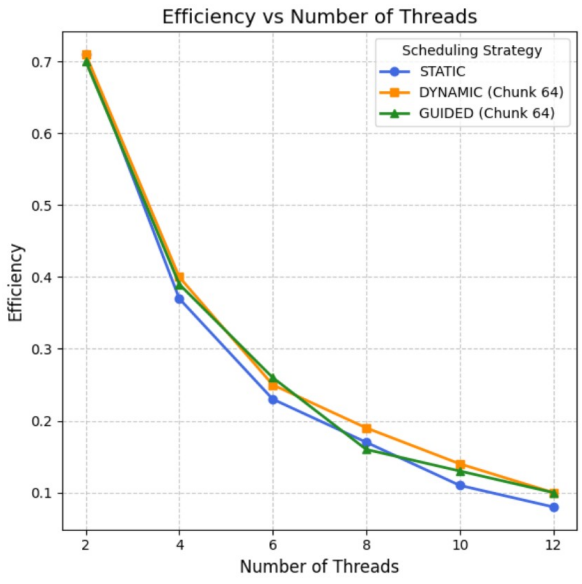
| Threads | Schedule Type | Execution Time (ms) | Speed Up | Throughput | Efficiency |
|---|---|---|---|---|---|
| | GUIDED (64) | 1562.76 | 1.56x | 1279.79 | 0.26 |
| **8** | STATIC | 1816.33 | 1.34x | 1101.12 | 0.17 |
| | DYNAMIC (64) | 1630.68 | 1.50x | 1226.49 | 0.19 |
| | GUIDED (64) | 1875.11 | 1.30x | 1066.61 | 0.16 |
| **10** | STATIC | 2136.95 | 1.14x | 935.91 | 0.11 |
| | DYNAMIC (64) | 1764.71 | 1.38x | 1133.33 | 0.14 |
| | GUIDED (64) | 1934.62 | 1.26x | 1033.79 | 0.13 |
| **12** | STATIC | 2406.65 | 1.01x | 831.03 | 0.08 |
| | DYNAMIC (64) | 2042.87 | 1.19x | 979.02 | 0.10 |
| | GUIDED (64) | 1992.70 | 1.22x | 1003.66 | 0.10 |

**OBSERVATION:**

A 2D heat diffusion simulation was parallelized over a 2000×2000 grid for 500 time steps using OpenMP, testing three scheduling strategies — Static, Dynamic (chunk 64), and Guided (chunk 64) — across thread counts from 2 to 12. The sequential baseline ran at 2438.33ms. Overall, the speedups were disappointingly low, peaking at only ~1.61× with 4 threads using Dynamic scheduling, and actually degrading below the sequential time at higher thread counts — Static with 12 threads achieved a mere 1.01× speedup. This is characteristic of a memory-bandwidth-bound stencil computation: the inner loop reads five neighboring cells per output cell, causing heavy cache pressure as threads compete for the same memory bus. Dynamic scheduling slightly outperformed Static at higher thread counts because it avoids idle threads when some rows take longer due to cache contention, while Guided performed inconsistently. Beyond 4–6 threads, adding more threads increases synchronization and memory contention faster than it adds compute, which explains the steady degradation in both speedup and efficiency.




Throughput vs Number of Threads
Scheduling Strategy
STATIC
DYNAMIC (Chunk 64)
GUIDED (Chunk 64)
Throughput
1300
1200
1100
1000
900
2
4
6
8
10
12
Number of Threads


Efficiency vs Number of Threads
Scheduling Strategy
STATIC
DYNAMIC (Chunk 64)
GUIDED (Chunk 64)
Efficiency
0.7
0.6
0.5
0.4
0.3
0.2
0.1
2
4
6
8
10
12
Number of Threads

**OBSERVATION FROM PERF:**

```
Performance counter stats for './outputs/heat':

          106,211      context-switches                 #    572.4 cs/sec  cs_per_second
            4,517      cpu-migrations                   #     24.3 migrations/sec  migrations_per_second
           31,425      page-faults                      #    169.4 faults/sec  page_faults_per_second
       185,554.21 msec task-clock                       #      5.3 CPUs  CPUs_utilized
       46,972,316      cpu_core/branch-misses/          #      0.4 %  branch_miss_rate         (71.70%)
   11,961,788,466      cpu_core/branches/               #     64.5 M/sec  branch_frequency     (71.70%)
  179,411,317,592      cpu_core/cpu-cycles/             #      1.0 GHz  cycles_frequency       (71.70%)
  140,256,900,473      cpu_core/instructions/           #      0.8 instructions  insn_per_cycle  (71.70%)
       32,000,329      cpu_atom/branch-misses/          #      0.6 %  branch_miss_rate         (17.32%)
    4,937,897,144      cpu_atom/branches/               #     26.6 M/sec  branch_frequency     (17.33%)
  147,637,485,792      cpu_atom/cpu-cycles/             #      0.8 GHz  cycles_frequency       (17.34%)
   56,102,764,760      cpu_atom/instructions/           #      0.4 instructions  insn_per_cycle  (17.35%)
            TopdownL1 (cpu_core)                        #      1.2 %  tma_bad_speculation
                                                        #      7.6 %  tma_frontend_bound       (71.70%)
                                                        #     70.3 %  tma_backend_bound
                                                        #     21.0 %  tma_retiring             (71.70%)
            TopdownL1 (cpu_atom)                        #     73.1 %  tma_backend_bound        (20.18%)
                                                        #      8.6 %  tma_frontend_bound       (20.18%)
                                                        #      2.2 %  tma_bad_speculation
                                                        #     16.1 %  tma_retiring             (20.18%)

     35.160416028 seconds time elapsed

    177.027267000 seconds user
      4.564602000 seconds sys
```

The program utilized 5.3 CPUs on average, but the IPC of only 0.8 on cpu_core is notably low, indicating the pipeline is frequently stalled waiting on data — a clear sign of a memory-bound workload. The branch miss rate is 0.4%, which is negligible. Most critically, the TopdownL1 analysis shows a severe 70.3% backend-bound on cpu_core and 73.1% on cpu_atom, the highest backend-bound figures seen across all experiments, confirming that memory latency from the stencil's scattered read pattern is the dominant bottleneck and the primary reason parallelism fails to yield meaningful speedup at higher thread counts.